



DR B. DIOP

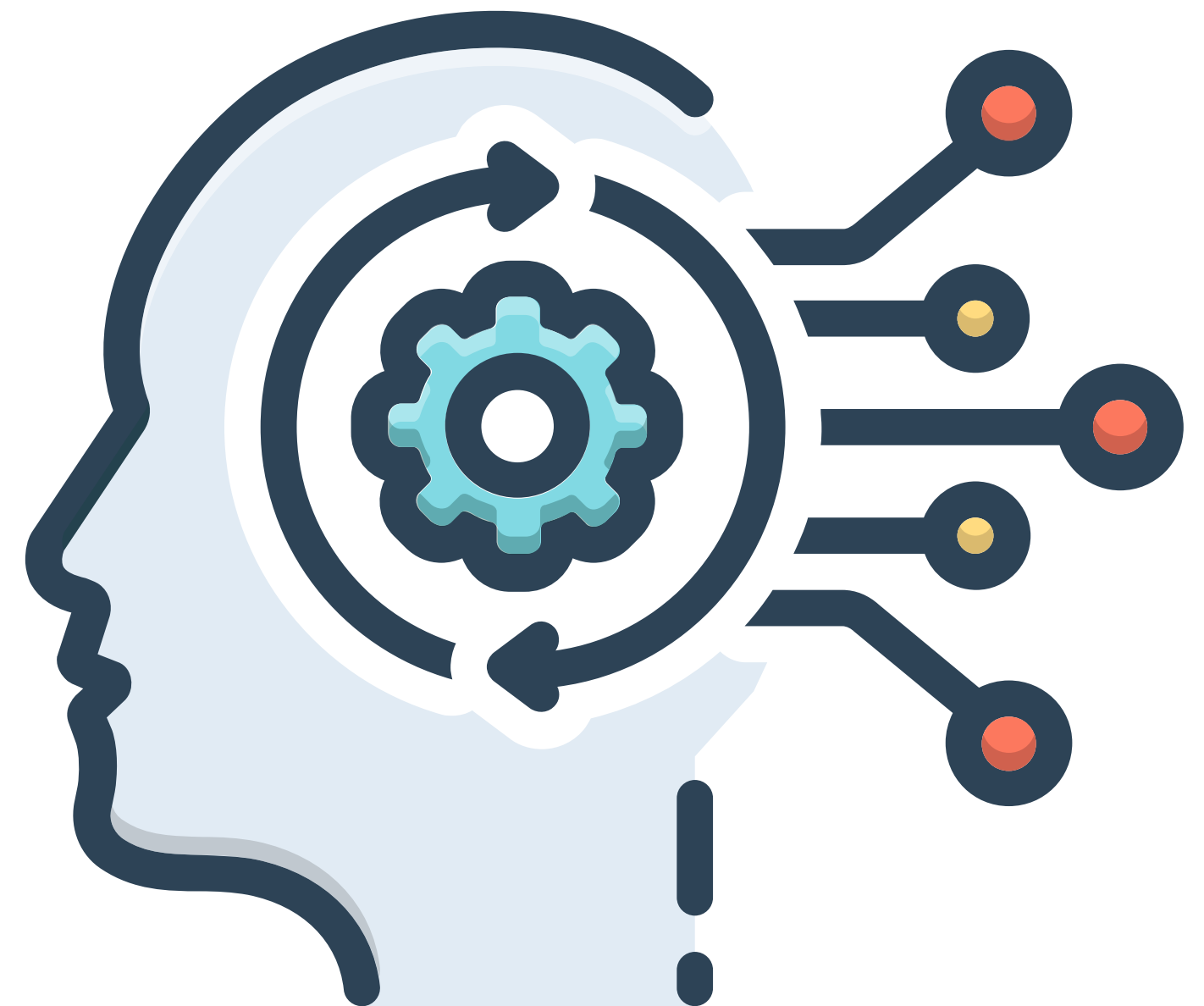
babacar.diop@ussein.edu.sn

Systemes distribués

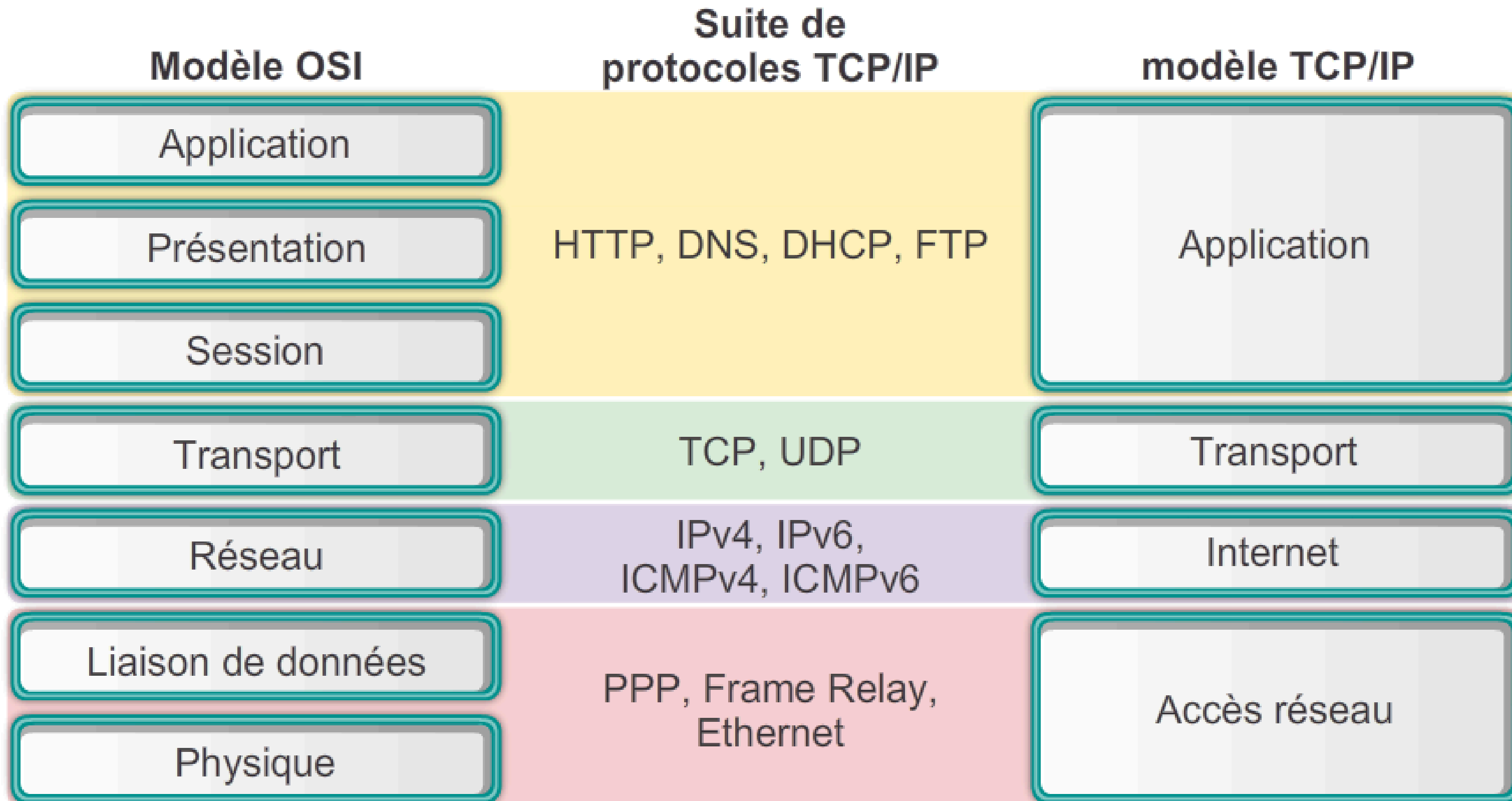
Sockets en TCP

INFO – AGROTIC

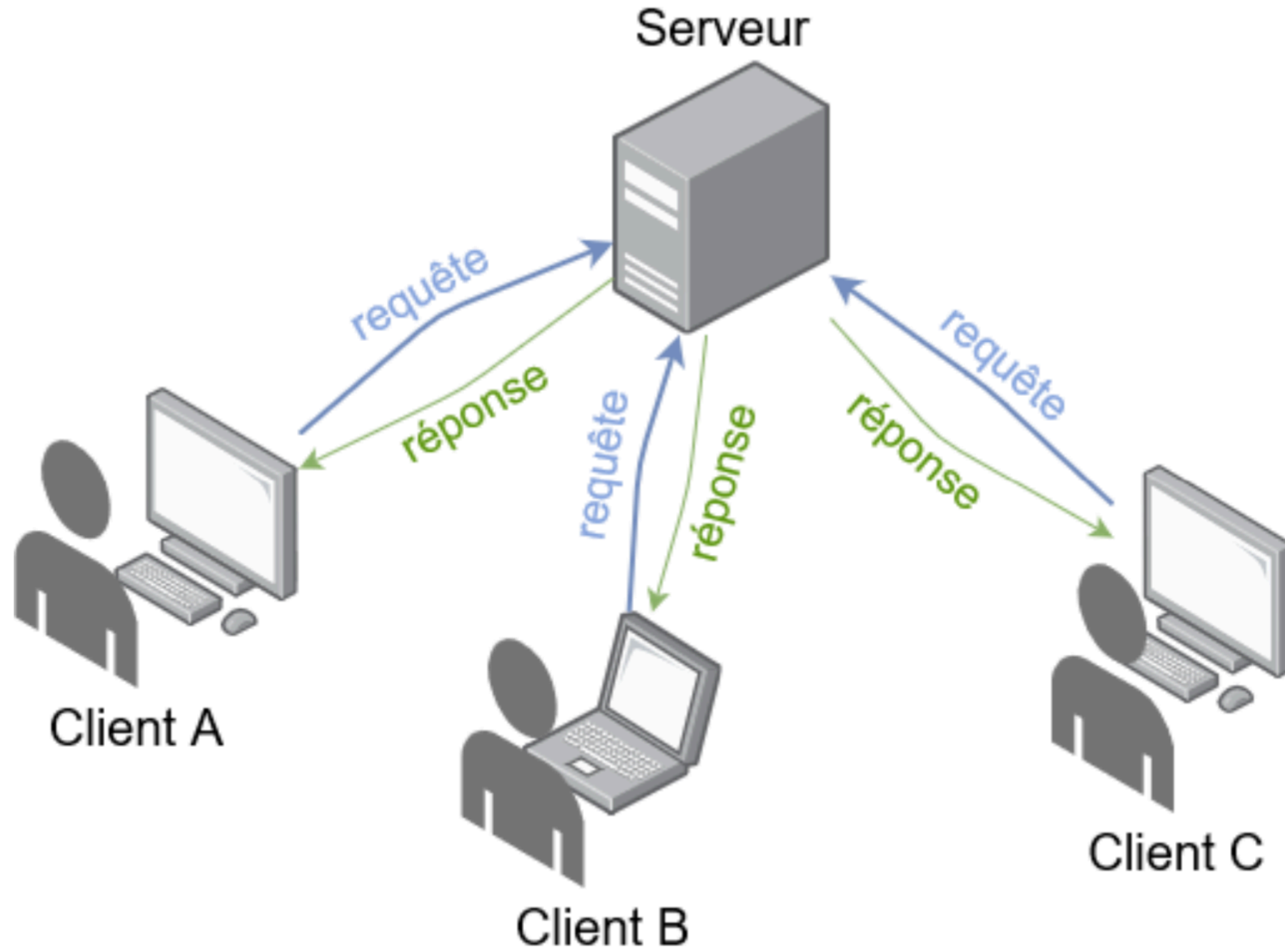
Last update: oct. 2025



Rappels modèle OSI



Rappels modèle OSI



Sockets - Historical Background

Origines

- 1971 : premières utilisations sur ARPANET.
- 1983 : intégration officielle dans BSD (Berkeley Software Distribution) → naissance de l'API Berkeley Sockets.

Sockets - Historical Background

Expansion with the Internet

- Années 1990 : explosion du World Wide Web.
- Les serveurs web, navigateurs et de nombreuses autres applications
- commencent à utiliser les sockets.
- Apparition massive du modèle client–serveur dans les applications réseau.

Sockets - Historical Background

Evolution & Stability

- Les protocoles sous-jacents ont évolué (TCP/IP, HTTP, etc.).
- Mais l'API bas niveau des sockets est restée pratiquement inchangée.

Sockets - Historical Background

Types of Socket Applications

- Client–Serveur : le serveur attend les connexions des clients.
- Internet sockets (Berkeley/BSD) : communication entre machines.
- Unix domain sockets : communication entre processus sur la même machine.

Python Socket API : Introduction

- Le module socket de Python fournit une interface vers l'**API Berkeley Sockets (BSD)**.
- Il permet la communication réseau bas niveau entre processus ou machines.
- **Fonctions principales :**
 - `socket()`,
 - `.bind()`,
 - `.listen()`,
 - `.accept()`
 - `.connect()`,
 - `.send()`,
 - `.recv()`,
 - `.close()`

Principales méthodes

Méthode	Rôle
socket()	Crée un nouvel objet socket
.bind()	Associe le socket à une adresse (IP + port)
.listen()	Met le socket en attente de connexions (serveur)
.accept()	Accepte une connexion entrante
.connect()	Demande une connexion (client)
.send() / .recv()	Envoi et réception de données
.close()	Ferme la connexion

Types de sockets

- **TCP (SOCK_STREAM)** : protocole par défaut
 - Fiable, connexion orientée
 - Livraison des données dans l'ordre
 - Réémission automatique en cas de perte
- **UDP (SOCK_DGRAM)** :
 - Non fiable, pas de connexion persistante
 - Données possibles hors ordre
 - Utilisé pour des communications rapides et légères
 -

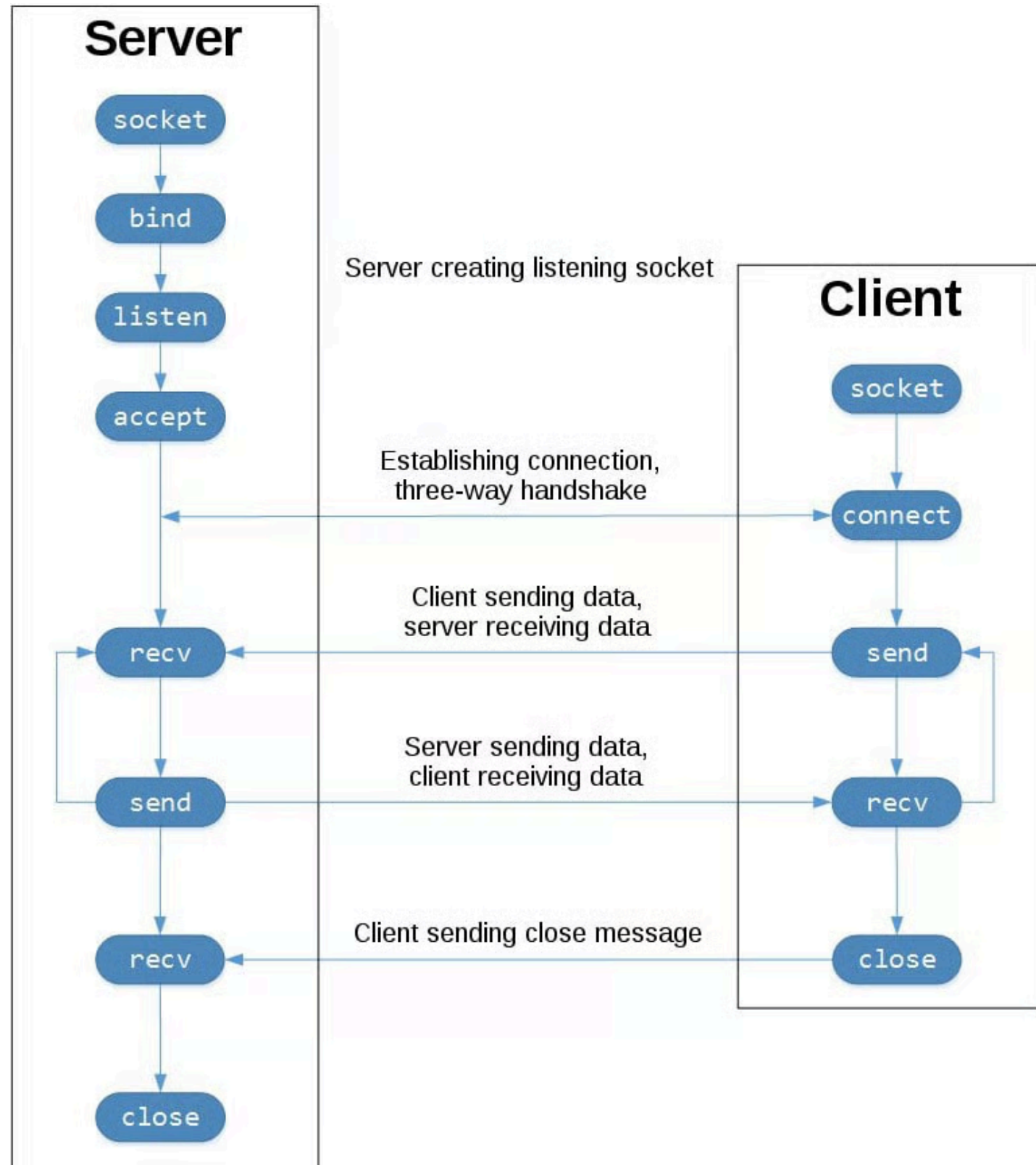
Pourquoi utiliser TCP ?

- Gère automatiquement :
 - la détection des pertes de paquets
 - la réémission et la séquence correcte des données
- Adapté aux applications nécessitant :
 - Fiabilité (ex. : messagerie, transfert de fichiers)
 - Ordre garanti

UDP est réservé aux cas :

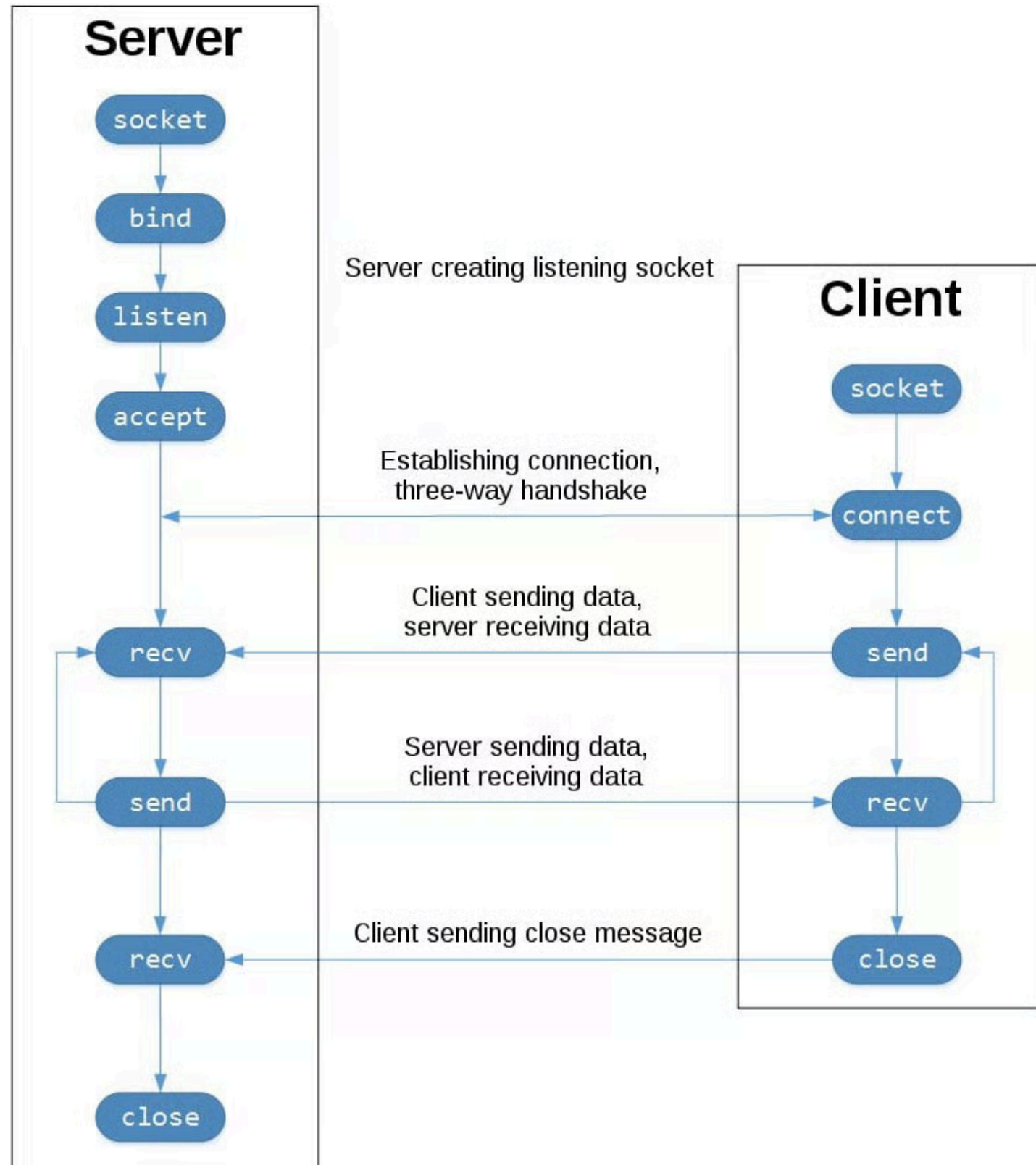
- Temps réel (streaming, jeux, IoT)
- Où quelques pertes sont acceptables

Flux de communication TCP (1/5)



- Le protocole TCP établit une connexion fiable client ↔ serveur.
- Le flux suit une séquence d'appels bien définie entre les deux côtés.
- Chaque étape assure la création, la communication, puis la fermeture de la connexion.

Flux de communication TCP (2/5)

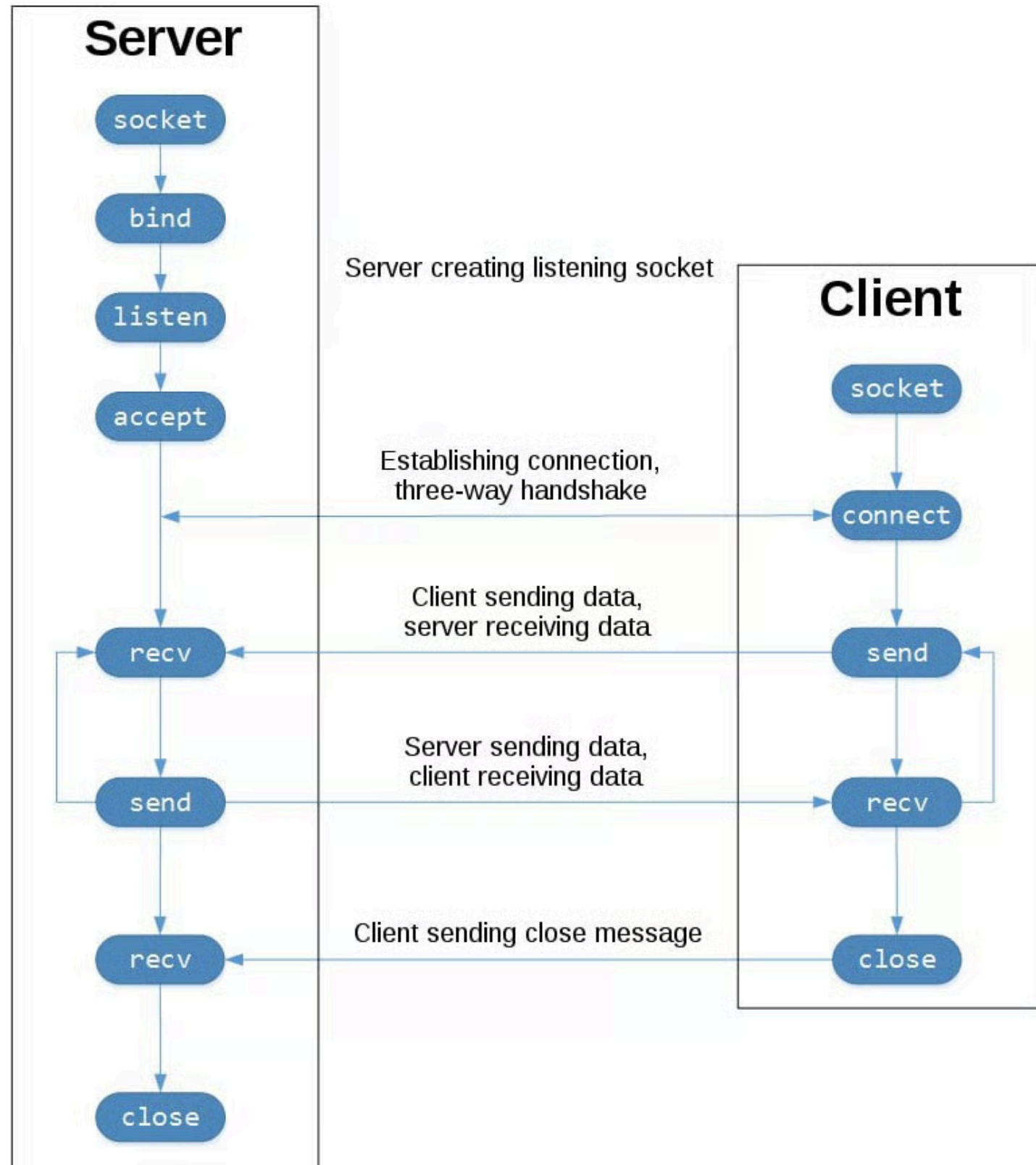


Côté serveur (gauche du schéma)

- **socket()** → création du socket
- **bind()** → associe une adresse IP + port
- **listen()** → met le socket en attente de connexions
- **accept()** → accepte une connexion client entrante

◆ Le socket devient alors une "porte d'entrée" écoutant les demandes.

Flux de communication TCP (3/5)

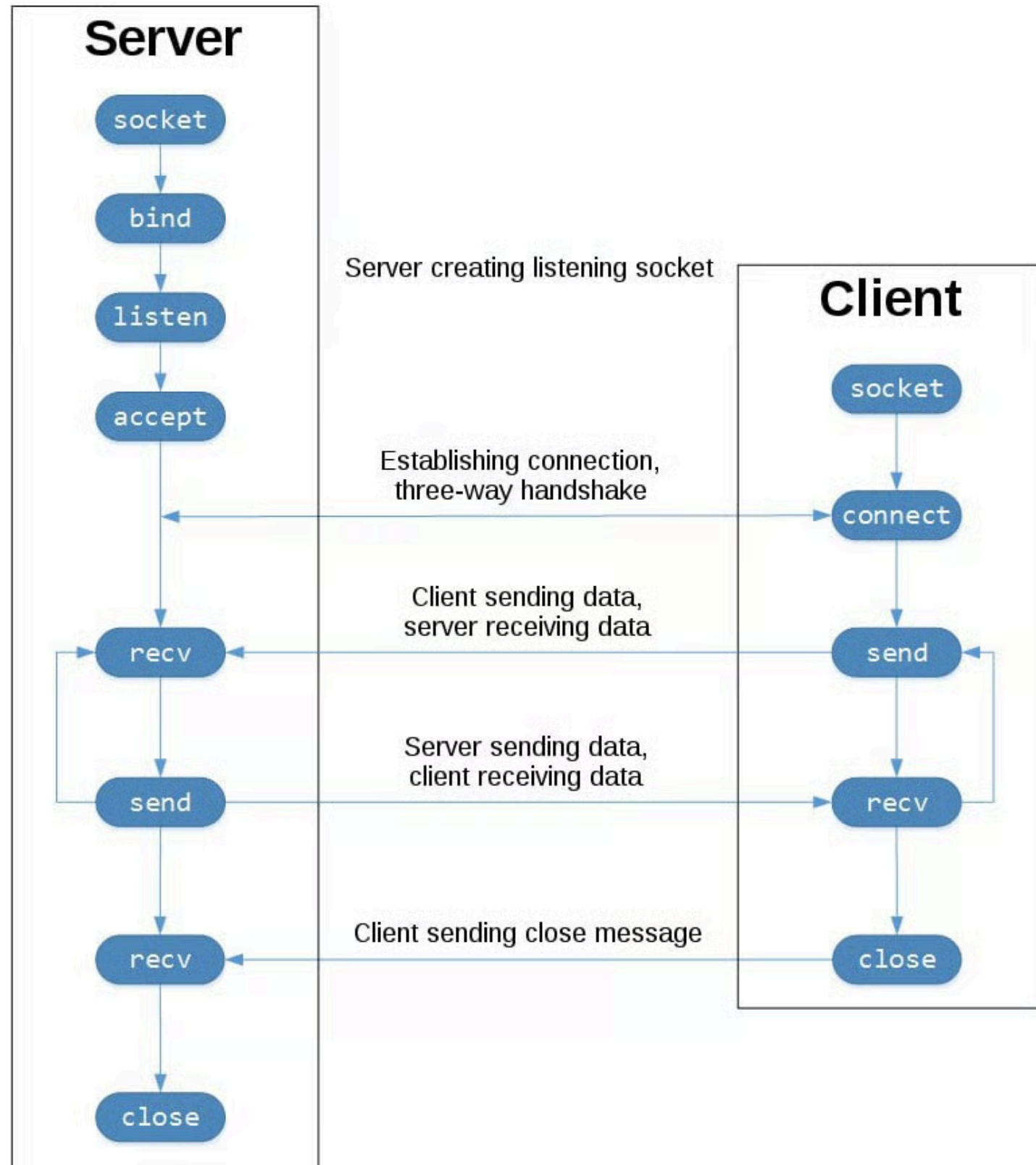


Côté client (droite du schéma)

- `socket()` → création du socket client
- `connect()` → demande de connexion vers le serveur
- Déclenche le **three-way handshake** TCP :
- `SYN` → `SYN-ACK` → `ACK`
- Vérifie que les deux hôtes sont joignables

◆ Une fois la connexion établie, le client et le serveur peuvent échanger des données.

Flux de communication TCP (4/5)



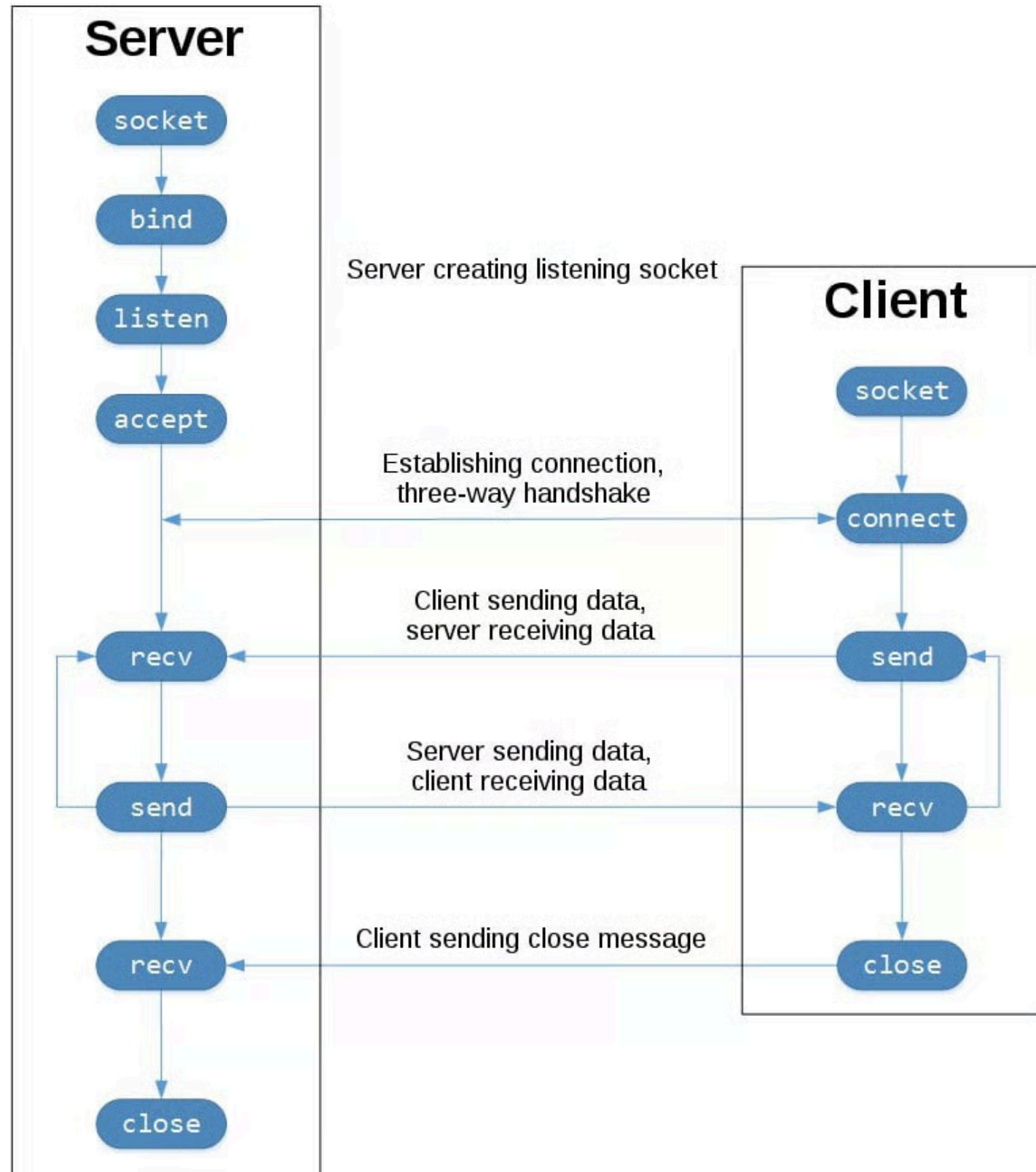
Échange de données

Une fois le serveur et le client connectés :

- **send()** : envoi des données par le client ou le serveur
- **recv()** : réception des données du côté opposé
- Le transfert est bidirectionnel et fiable.

◆ TCP garantit que les données arrivent dans l'ordre et sans perte.

Flux de communication TCP (5/5)



Fermeture de la connexion

- Une fois les échanges terminés :
 - Le client appelle `.close()`
 - Le serveur appelle également `.close()`
- Chaque côté libère ses ressources réseau.

◆ Fermeture propre = fin du flux TCP.

Echo client-serveur

- Maintenant que vous avez une vue d'ensemble de l'API socket et de la manière dont le client et le serveur communiquent, vous êtes prêt à créer votre premier client et serveur.
- Vous commencerez par une implémentation simple. Le serveur se contentera de renvoyer au client tout ce qu'il reçoit.

Echo client-serveur

Implémentation simple

Le serveur se contentera de renvoyer au client tout ce qu'il reçoit.

Python

echo-server.py

```
import socket

HOST = "127.0.0.1" # Standard loopback interface address (localhost)
PORT = 65432 # Port to listen on (non-privileged ports are > 1023)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)
```

Objectif : Echo Server

- Importer le socket sur Python
- Serveur écoute sur **127.0.0.1** et utilise un port > 1023 (ex. 65432).
- Utilisation du context manager (with socket.socket(...) as s) → fermeture automatique.

Python

echo-server.py

```
import socket
```

```
HOST = "127.0.0.1" # Standard loopback interface address (localhost)
```

```
PORT = 65432 # Port to listen on (non-privileged ports are > 1023)
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
```

Création du socket

- `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
- `AF_INET` → famille d'adresses IPv4
- `SOCK_STREAM` → TCP (fiable, orienté connexion)

Python

echo-server.py

```
import socket

HOST = "127.0.0.1" # Standard loopback interface address (localhost)
PORT = 65432 # Port to listen on (non-privileged ports are > 1023)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
```

Liaison et écoute

- **bind((HOST, PORT))** → associe le socket à une interface réseau et un port
- **127.0.0.1** → seulement localhost
- **Port** : 1–65535 (>1023 sans privilège root)
- **listen(backlog)** → rend le socket écoutant, prêt à accepter des connexions
- **backlog** : nombre maximum de connexions en attente

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
    s.bind((HOST, PORT))  
    s.listen()
```


Acceptation d'une connexion

- **accept()** → bloque jusqu'à ce qu'un client se connecte
- Retourne :
 - **conn** → nouveau socket pour communiquer avec ce client
 - **addr** → adresse du client (IP, port)
- **Important** : ce nouveau socket est distinct du socket qui écoute.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
    s.bind((HOST, PORT))  
    s.listen()  
    conn, addr = s.accept()
```

Échange de données

Boucle infinie avec :

```
while True:
    data = conn.recv(1024)
    if not data:
        break
    conn.sendall(data)
```

- Fonctionnement :
 - **recv()** → lecture des données envoyées par le client
 - **sendall()** → renvoie les données au client (echo)
 - Fin de boucle si le client ferme la connexion (recv() retourne b'')

Fermeture automatique

- L'utilisation de “**with conn:**” assure la fermeture automatique du socket client à la fin du bloc
- Permet d'éviter d'appeler explicitement **conn.close()**.
- Le serveur peut ensuite accepter une nouvelle connexion.

```
with conn:
    print(f"Connected by {addr}")
    while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data)
```

client TCP (Echo Client)

Objectif :

- Comprendre comment un programme client se connecte à un serveur TCP, envoie un message et reçoit une réponse.
- **Principe :**
 - Le client
 - → initie la connexion
 - → envoie un message
 - → attend la réponse du serveur.

client TCP (Echo Client)

Python

echo-client.py

```
import socket

HOST = "127.0.0.1" # The server's hostname or IP address
PORT = 65432 # The port used by the server

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello, world")
    data = s.recv(1024)

print(f"Received {data!r}")
```

Le code du client

Voici comment un programme client se connecte à notre serveur TCP, envoie un message et reçoit une réponse.

Le client envoie une chaîne "Hello, world" au serveur et affiche la réponse reçue.

Étapes principales coté client

Étape	Appel de fonction	Description
	<code>socket.socket()</code>	Crée la socket TCP
	<code>connect()</code>	Se connecte au serveur
	<code>sendall()</code>	Envoie le message
	<code>recv(1024)</code>	Lit la réponse du serveur
	<code>print()</code>	Affiche la réponse

Exécution pratique

Démarrer le serveur :

```
python echo-server.py
```

Le serveur reste en attente sur **.accept()**
(il “écoute” les connexions entrantes)

```
HOST = "127.0.0.1" # Standard loopback
PORT = 65432 # Port to listen on (non-l

with socket.socket(socket.AF_INET, socke
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)
```


Exécution pratique

Démarrer le client :

```
python echo-client.py
```

Résultat côté client :

```
Reçu : b'Hello, world'
```

Python

echo-client.py

```
import socket

HOST = "127.0.0.1" # The server's hostname or IP address
PORT = 65432 # The port used by the server

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello, world")
    data = s.recv(1024)

print(f"Received {data!r}")
```

Exécution pratique

Résultat côté serveur :

```
Connected by ('127.0.0.1', 64623)
```

Résultat côté client :

```
Reçu : b'Hello, world'
```

**Visualisation du flux de communication
une fois la connexion établie :**

[CLIENT]		[SERVEUR]
	---- connect() ----->	
	---- "Hello, world" ----->	
	<--- "Hello, world" -----	

Problème des Serveurs Bloquants

Le serveur traditionnel

```
while True:
    conn, addr = sock.accept()    # BLOQUE ici !
    data = conn.recv(1024)        # BLOQUE ici aussi !
    conn.send(data)
```

Conséquences :

- ✗ Un seul client à la fois
- ✗ Les autres clients attendent
- ✗ Le serveur "gèle" pendant les opérations I/O
- ✗ Inefficace et non scalable

Solution : Mode Non-Bloquant + Selectors

Principe de base - Mode non-bloquant :

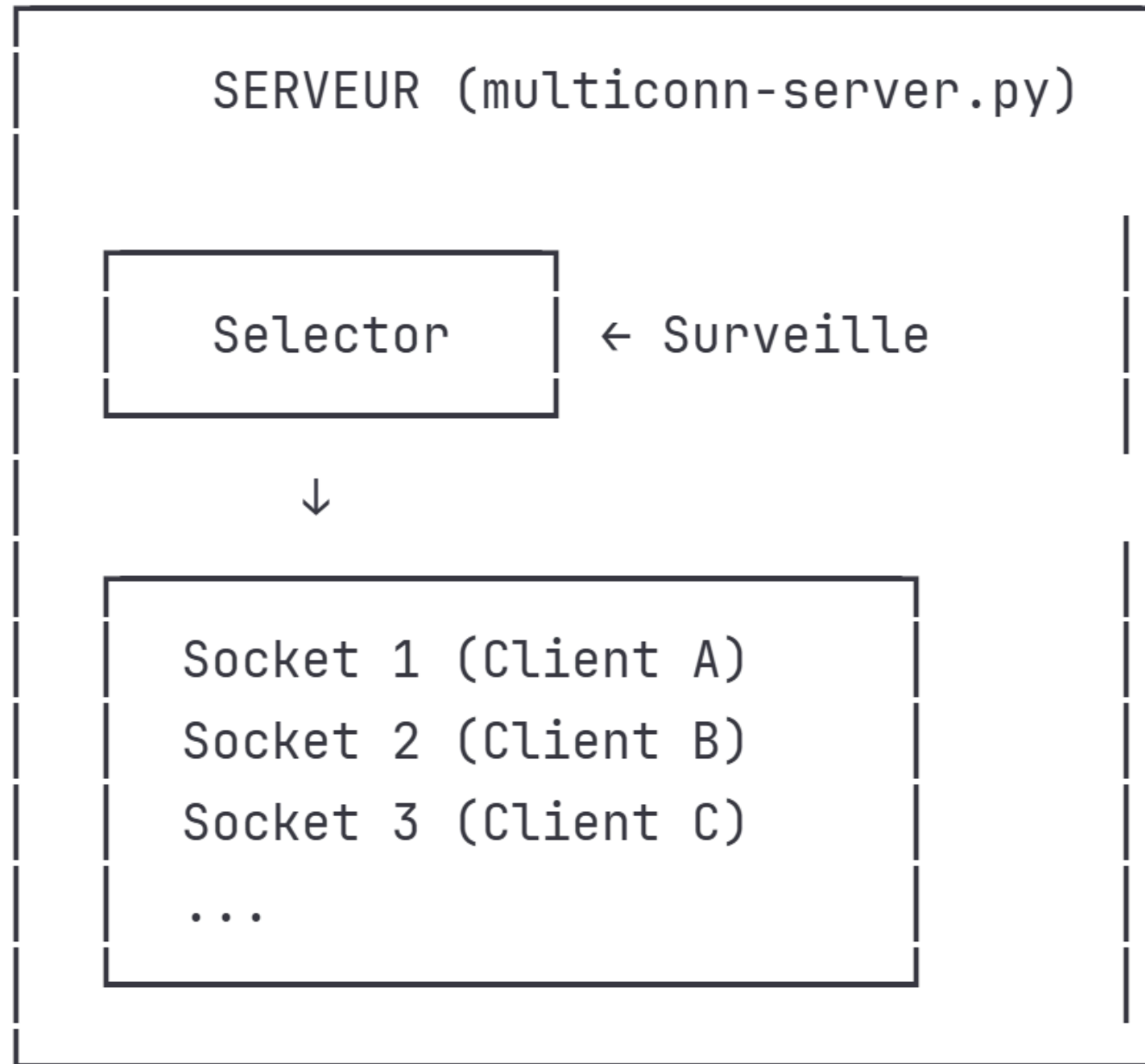
```
sock.setblocking(False)  # Le socket ne bloque plus !
```

Selector :

- Surveille plusieurs sockets simultanément
- Notifie quand un socket est prêt (lecture/écriture)
- Permet de gérer 100, 1000+ clients en parallèle

Analogie : Un chef d'orchestre qui coordonne plusieurs musiciens

Vue d'ensemble de l'architecture



Serveur multi connexions (1/4)

Initialisation

```
import selectors
import socket

sel = selectors.DefaultSelector()

# Socket d'écoute
lsock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
lsock.bind((host, port))
lsock.listen()
lsock.setblocking(False) # ← MODE NON-BLOQUANT

# Enregistrement dans le selector
sel.register(lsock, selectors.EVENT_READ, data=None)
```



Point clé : **data=None**
identifie le socket
d'écoute

Serveur multi connexions (2/4)

Boucle d'événement

```
try:
    while True:
        events = sel.select(timeout=None) # Attend des événements

        for key, mask in events:
            if key.data is None:
                # Nouvelle connexion
                accept_wrapper(key.fileobj)
            else:
                # Client existant
                service_connection(key, mask)

except KeyboardInterrupt:
    print("Arrêt du serveur")
finally:
    sel.close()
```

bloque jusqu'à ce qu'un
socket soit prêt pour une
opération d'I/O et renvoie
une liste de tuples
(key, mask)

key.fileobj: Le socket concerné

mask : type d'opération prête
(lecture/écriture)

Serveur multi connexions (3/4)

Acceptation des connexions

```
def accept_wrapper(sock):
    conn, addr = sock.accept()
    print(f"Connexion acceptée de {addr}")

    conn.setblocking(False)  # Client en mode non-bloquant

    # Données associées au client
    data = types.SimpleNamespace(
        addr=addr,
        inb=b'',  # Buffer d'entrée
        outb=b''  # Buffer de sortie
    )

    # Surveillance lecture ET écriture
    events = selectors.EVENT_READ | selectors.EVENT_WRITE
    sel.register(conn, events, data=data)
```

Serveur multi connexions (4/4)

Traitement des clients

```
def service_connection(key, mask):  
    sock = key.fileobj  
    data = key.data  
  
    # Le socket est prêt pour la LECTURE  
    if mask & selectors.EVENT_READ:  
        recv_data = sock.recv(1024)  
        if recv_data:  
            data.outb += recv_data # Stocker pour renvoyer  
        else:  
            # Client déconnecté  
            sel.unregister(sock)  
            sock.close()  
  
    # Le socket est prêt pour l'ÉCRITURE  
    if mask & selectors.EVENT_WRITE:  
        if data.outb:  
            sent = sock.send(data.outb)  
            data.outb = data.outb[sent:] # Supprimer envoyé
```

Client Multi-Connexions (1/3)

Création des connexions

```
def start_connections(host, port, num_conns):
    server_addr = (host, port)

    for i in range(num_conns):
        connid = i + 1
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.setblocking(False)

        # connect_ex() ne bloque pas !
        sock.connect_ex(server_addr)

        data = types.SimpleNamespace(
            connid=connid,
            messages=[b"Message 1", b"Message 2"],
            outb=b"",
            recv_total=0
        )

        sel.register(sock, EVENT_READ | EVENT_WRITE, data=data)
```


Client Multi-Connexions (2/3)

Envoi des messages

```
if mask & selectors.EVENT_WRITE:
    # Prendre le prochain message
    if not data.outb and data.messages:
        data.outb = data.messages.pop(0)

    # Envoyer
    if data.outb:
        print(f"Envoi {data.outb!r} connexion {data.connid}")
        sent = sock.send(data.outb)
        data.outb = data.outb[sent:]
```

Réception et fermeture

Réception et fermeture

```
if mask & selectors.EVENT_READ:
    recv_data = sock.recv(1024)
    if recv_data:
        print(f"Reçu {recv_data!r} connexion {data.connid}")
        data.recv_total += len(recv_data)

    # Fermeture si tout reçu
    if not recv_data or data.recv_total == data.msg_total:
        print(f"Fermeture connexion {data.connid}")
        sel.unregister(sock)
        sock.close()
```

Flux d'exécution détaillé

1. Serveur écoute sur port 65432
↳ `sel.select()` attend...
2. Client A se connecte
↳ `sel.select()` retourne `EVENT_READ` sur `lsock`
↳ `accept_wrapper()` crée connexion A
↳ Connexion A enregistrée dans `selector`
3. Client B se connecte
↳ Même processus → Connexion B enregistrée
4. Client A envoie "Message 1"
↳ `sel.select()` retourne `EVENT_READ` sur `conn A`
↳ `service_connection()` lit et stocke dans `outb`
5. Connexion A prête pour écriture
↳ `sel.select()` retourne `EVENT_WRITE` sur `conn A`
↳ `service_connection()` renvoie "Message 1"
6. Même temps : Client B envoie aussi !
↳ Pas de blocage, traité dans la même boucle

Mise en évidence des avantages

Aspect	Serveur Bloquant	Serveur Multi-Connexions
Clients simultanés	1 seul	100+
Blocage I/O	Oui ❌	Non ✅
Performance	Faible	Élevée
Complexité	Simple	Moyenne
Scalabilité	Aucune	Excellente

Lancement du serveur

```
$ python multiconn-server.py 127.0.0.1 65432
```

```
Listening on ('127.0.0.1', 65432)
```

```
Accepted connection from ('127.0.0.1', 61354)
```

```
Accepted connection from ('127.0.0.1', 61355)
```

```
Echoing b'Message 1...' to ('127.0.0.1', 61354)
```

```
Echoing b'Message 1...' to ('127.0.0.1', 61355)
```


Lancement du client

```
$ python multiconn-client.py 127.0.0.1 65432 2
```

```
Starting connection 1 to ('127.0.0.1', 65432)
```

```
Starting connection 2 to ('127.0.0.1', 65432)
```

```
Sending b'Message 1...' to connection 1
```

```
Sending b'Message 1...' to connection 2
```

Concepts clés à retenir

- **setblocking(False)** : Empêche le blocage des opérations I/O
- **Selector** : Multiplexeur d'événements I/O
- **EVENT_READ** : Socket prêt à lire
- **EVENT_WRITE** : Socket prêt à écrire
- **Buffers (inb/outb)** : Stockage temporaire des données
- **Unregister** : Retirer un socket du selector avant fermeture

Exercices pratiques

- **Basique** : Modifier le serveur pour compter les messages reçus
- **Intermédiaire** : Ajouter un timeout de 30s par connexion
- **Avancé** : Créer un serveur de chat multi-clients
- **Projet** : Implémenter un protocole personnalisé (JSON, headers HTTP)

